Available online at www.sciencedirect.com

ScienceDirect

Procedia Computer Science 183 (2021) 471–478

Procedia Computer Science

www.elsevier.com/locate/procedia

10th International Conference of Information and Communication Technology (ICICT-2020)

Optimization of workpiece detection model based on channel pruning and fixed-point quantization

Bin Chai[a,b,*], Weiguang Han[a,b], Lanqing Ye[a,b]

[a]*Shenyang Institute of Computing Technology, Chinese Academy of Sciences, Shenyang, 110168, China*
[b]*University of Chinese Academy of Sciences, Beijing, 100049, China*

Abstract

As the development of deep learning, the accuracy and speed of object detection tasks are constantly improved, and more and more neural network models meet the needs of object detection. Many network models provide a new possibility for workpiece detection in factory workshops. However, the existing object detection model has the problems of large parameters, slow running speed and high memory consumption, which makes it difficult to be applied to the actual workshop production scene. In order to solve the above problems, based on the research of network model compression and acceleration algorithm, this paper optimizes the classical mobile Mobilenet-yolov3 network by quantification and pruning, and makes experimental analysis and comparison with existing result. The experimental results show that using the same workpiece data set, the new model can get the same effect in F1 score and accuracy rate as the original model, and also slightly improve the recall rate. Compared with the original model, the computational complexity and model size and the Flops (forward inference speed) are greatly reduced.

Keywords: Workpiece detection; model compression; model acceleration; pruning; quantification;

1. Introduction

In recent years, deep neural network models have penetrated into various fields of computer vision. Object detection is one of the typical representatives, which is widely used in security monitoring, face recognition, license plate detection and other fields. In the field of industrial vision, there are still many bottlenecks in the

* Corresponding author. Tel.:+86-137-1779-8912.
*E-mail address:*1095135384@qq.com.

10.1016/j.procs.2021.02.086

implementation of neural network models. From the perspective of factory production scenes, industrial scenes are simpler than life scenes, and the object tasks are fixed, which reduces the difficulty of workpiece object tasks to a certain extent, but industrial scenes have different requirements for object speed and accuracy. In addition, most of the current mainstream neural network models are trained and tested on the PC side or the server side. The parameter storage and inference computing capabilities of their models can be greatly satisfied. However, the edge devices of the industrial scene haven't enough computing and storage capabilities. In order to meet the needs of actual industrial scenarios, it is necessary to perform model compression and acceleration[8] on existing models, and reduce the size and operating speed of the model without excessive loss of accuracy. To this end, this article explores two common model compression and acceleration strategies, fixed-point quantization and channel uniform pruning, and applies them to the classic MobileNet-Yolov3. Experiments have proved that after fixed-point quantization and uniform pruning of channels, the model size is greatly reduced, the model runs faster, and the model prediction accuracy is not lost too much.

2. Introduction to MobileNet-Yolov3 model

2.1. Yolov3 module

Yolov3 network[1] is a classic single-stage object detection network, and its network structure is shown in Figure 1:

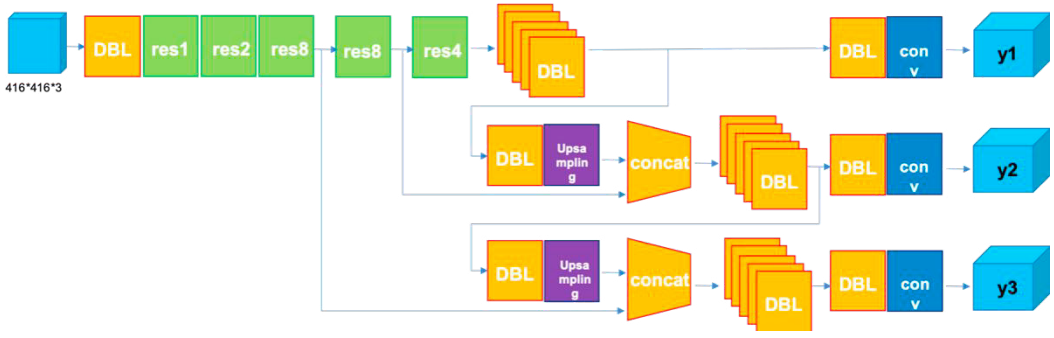

Fig. 1. YOLOv3 network structure diagram.

The core structure of YOLOv3 can be simplified into two parts: "feature extraction backbone network + multi-scale feature fusion". Its core components mainly include the DBL (conv+BN+Leaky relu) part, and superimposes the Res_unit using the Resblock_body part. Residual connection enables the model to perform more feature fusion, avoiding the loss of feature information as the network depth increases. The YOLOv3 network is very suitable for predicting small targets. It designs three-scale independent predictions in the final prediction branch and merges the three prediction results. The feature layer sizes of the three prediction branches are 52, 26, and 13, respectively.

2.2. MobileNet module

Compared with other feature extraction networks, the innovation of MobileNet[2] network is the introduction of depthwise convolution. The depthwise separable convolution integrates the standard convolution into two steps: the first step is Depthwise convolution, that is, channel-by-channel convolution. One convolution kernel is responsible for one channel, and one channel is "filtered" by only one convolution kernel; the second step , PointWise convolution, "connect" the results obtained by Depthwise convolution.

The core convolution extraction module plays a key role in simplifying the parameters of the convolution kernel. YOLOv3 uses darknet53 as its backbone network in order to improve the flexibility of its network model. The backbone network of YOLOv3 can also be replaced with a variety of convolutional networks with feature extraction. The MobileNet-Yolov3 network essentially replaces the backbone network with the MobileNet network, which can

greatly reduce the amount of network parameters and calculations in the backbone network. Using MobileNet to replace large networks is also a common technique for edge deployment of models.

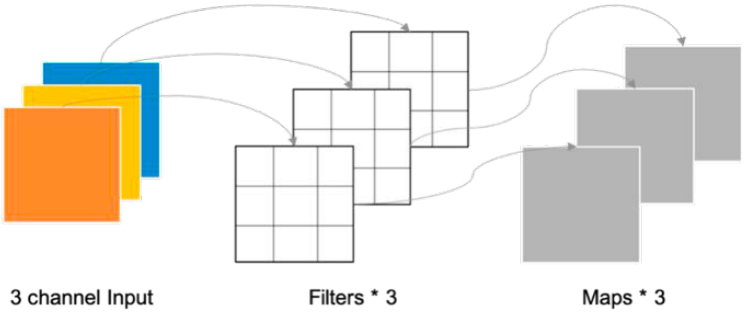

Fig. 2. Depth separable convolution.

3. Fixed-point quantization and channel pruning

3.1. Fixed-point quantization

Quantization[6,7] is to change the storage and representation format of model network parameters from high-precision type to low-precision type. The most commonly used is to change the FP32 format to FP16 or INT8 data types. Replacing high-precision floating-point types with low-precision integer types can be called fixed-point quantization. Practice has proved that the use of fixed-point quantization can greatly compress the model and increase the memory bandwidth, and efficiently use the cache (many hardware devices, memory access is the main energy consumption). In addition, quantization can speed up the calculation speed of the model (usually with an increase of 1x-3x). Currently commonly used quantization methods are mainly divided into Post Training Quantization and Quantization Aware Training, both of which support fixed-point quantization. The difference is that the former is the operation of model quantization after the model is trained, while the latter is the introduction of the idea of fake quantization, which is trained together with the original model, also known as "training quantization". Compared with the first method, "training quantization" has better quantization accuracy and it's more suitable for small object detection tasks such as workpiece detection. So this article uses "training quantization" to perform fixed-point quantization operations.

The general mathematical description of converting floating-point numbers into fixed-point numbers is as follows,

$$r = \min(\max(x, a), b) \tag{1}$$

$$s = \frac{b-a}{n-1} \tag{2}$$

$$q = \left\lfloor \frac{r-a}{s} \right\rceil \tag{3}$$

In the formula, x is the floating-point value to be quantized; [a, b] is the quantization range; a is the minimum value of the floating-point number to be quantized; b is the maximum value of the floating-point number to be quantized. [] means round the result to the nearest integer. If the quantization level is k, n is 2^k. For example, if k is 8, then n is 256. q is an integer obtained by quantization. This article chooses the maximum absolute value quantization (max-abs) method, which is described as follows:

$$M = \max(abs(x)) \tag{4}$$

$$q = \left\lfloor \frac{x}{M} * (n-1) \right\rceil \tag{5}$$

In the formula, x is the floating-point value to be quantized; M is the maximum absolute value of the quantized floating-point number; [] indicates that the result is rounded to the nearest integer.

3.2. Fixed-point quantitative training

Quantitative training is mainly divided into forward propagation part and backpropagation training part. Forward propagation and reverse training can be carried out according to the following steps:

(1) The input and weight are quantized into 8-bit integers according to the fixed-point quantization method above.

(2) Perform general matrix operations on 8-bit integers. The specific formula is as follows.

$$X_{\mathrm{q}} = \left\lfloor \frac{X}{X_{\mathrm{m}}} * (n-1) \right\rceil \tag{6}$$

$$W_{\mathrm{q}} = \left\lfloor \frac{W}{W_{\mathrm{m}}} * (n-1) \right\rceil \tag{7}$$

Perform general matrix operations, the calculation formula is as follows.

$$Y_q = X_q * W_q \tag{8}$$

(3) The output result of dequantization general matrix multiplication is 32-bit floating point data, and the dequantization formula is as follows.

$$\begin{aligned} Y_{\mathrm{dq}} &= \frac{Y_q}{(n-1)*(n-1)} * X_m * W_m \\ &= \left(\frac{X_q}{n-1} * X_m\right) * \left(\frac{W_{\mathrm{q}}}{n-1}\right) * W_m \end{aligned} \tag{9}$$

(4) Execute offset addition operation on 32-bit floating point data. The bias has not been quantified at this time.

(5) The reverse training part is mainly the calculation and update of the weight gradient value. The update of the weight gradient value can be obtained through the quantized weight and the quantized activation. All inputs and outputs of the backpropagation process are 32-bit floating point data. It should be noted here that the gradient update operation needs to be performed on the original weight, that is, the calculated gradient will be added to the original weight instead of the quantized or dequantized weight.

3.3. Uniform channel pruning

In academia, common pruning ideas include uniform channel pruning[4,5], convolution channel pruning[9,10] based on sensitivity and automatic pruning based on evolutionary algorithms. The latter two kinds of pruning are suitable for difficult detection tasks, more targets, more complex network structures, and the pruning results are not intuitive enough. Therefore, this article adopts the first uniform channel pruning method for pruning. "Uniform channel trimming" is to trim the convolution kernel of the fixed channel at a constant proportion. The trimming method is intuitive and violent, but it saves a lot of hyperparameter adjustment and the effect is more effective. Since the backbone network MobileNet in this article is a feature extraction network thinned by a convolution kernel, it is relatively lightweight and theoretically does not need to be tailored. Therefore, the focus of tailoring is placed on the multi-scale feature fusion part of YOLOv3. In this paper, the 16yolo_block_conv used in the multi-scale feature fusion network is uniformly cropped. The three branches of the YOLOv3 multi-scale fusion are reduced in sequence according to the size of the detection target, the size of the convolution kernel, and the corresponding convolution

kernel parameters. So you can consider more tailoring for the large convolution kernel when cropping, and less tailoring for the convolution kernel with fewer parameters. In this article, the convolution kernels of the three branches are respectively 0.5, 0.4, 0.3.The uniform cut is as follows:

The first column is the name of the convolution kernel to be clipped, the second column is the shape corresponding to the original convolution kernel, and the third column is the uniform clipping rate of the convolution kernel.

Table 1. Channel pruning rate.

Parameter	Shape	Prune ratiao
Yolo_block.0.0.0.conv.weights	(512, 1024, 1, 1)	0.5
Yolo_block.0.0.1.conv.weights	(1024, 512, 3, 3)	0.5
Yolo_block.0.1.0.conv.weights	(512, 1024, 1, 1)	0.5
Yolo_block.0.1.1.conv.weights	(1024, 512, 3, 3)	0.5
Yolo_block.0.2.conv.weights	(512, 1024, 1, 1)	0.5
Yolo_block.0.tip.conv.weights	(1024, 512, 3, 3)	0.5
Yolo_block.1.0.0.conv.weights	(256, 768, 1, 1)	0.4
Yolo_block.1.0.1.conv.weights	(512, 256, 3, 3)	0.4
Yolo_block.1.1.0.conv.weights	(256, 512, 1, 1)	0.4
Yolo_block.1.1.1.conv.weights	(512, 256, 3, 3)	0.4
Yolo_block.1.2.conv.weights	(256, 512, 1, 1)	0.4
Yolo_block.1.tip.conv.weights	(512, 256, 3, 3)	0.4
Yolo_block.2.0.0.conv.weights	(128, 384, 1, 1)	0.3
Yolo_block.2.0.1.conv.weights	(256, 128, 3, 3)	0.3
Yolo_block.2.1.0.conv.weights	(128, 256, 1, 1)	0.3
Yolo_block.2.1.1.conv.weights	(256, 128, 3, 3)	0.3
Yolo_block.2.2.conv.weights	(128, 256, 1, 1)	0.3
Yolo_block.2.tip.conv.weights	(256, 128, 3, 3)	0.3

4. Experiment analysis

The experimental environment in this paper is a cloud server with 4 core CPU, RAM 32GB, GPU v100, 16GB video memory, and 100GB disk. The Two-class workpiece data set is used as the experimental data set, and the five indicators of model size, Flops, F1 score, Mean_Precision and Mean_Recall are used to measure the pruning quantitative effect of the workpiece detection model.

4.1. No pruning and no quantification workpiece detection experiment

Using the Two-class workpiece data as a data set, divide it into a training set and a test set at a ratio of 8:2, and perform model training on the MobileNet-Yolov3 network and use it on the test set for prediction. The prediction effect is shown in Figure 3 above.

It can be seen from Figure 3 that, except for some targets with oblique angles and severe crossovers, all other targets can be correctly detected and positioned. It shows that in this data set, the detection effect of the MobileNet-Yolov3 model is better. The summary indicators and F1 score curve are shown below. It can be seen from the figure that after training for 10 epochs, the model has reached an over-fitting state, indicating that the MobileNet-Yolov3 model can achieve the expected error after training for 10 epochs under the current data set noise lower bound. Therefore, in the later pruning and quantization experiments, training 10epoch is used as the base of the training cycle, which greatly reduces the parameter adjustment range and reduces the experiment time.

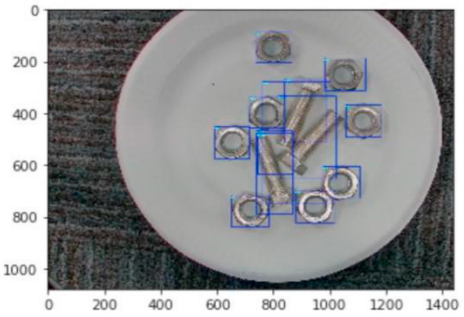

Fig. 3. Workpiece detection effect map.

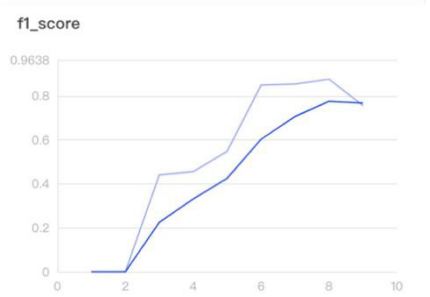

Fig. 4. Original model F1 score result graph.

4.2. There are pruning and quantitative experiments

After the MobileNet-Yolov3 model is subjected to fixed-point quantization and channel uniform pruning operations, it is trained according to the number of cycles of [1,3,5,7,9,10]. The image detection results under different quantization training cycles are shown in Figure 5.

The curves of the F1 score of the best accuracy rate for the first category, the best accuracy rate for the second category, the best recall rate for the first category, and the best recall rate for the Two-category are shown in Figure 6 below.

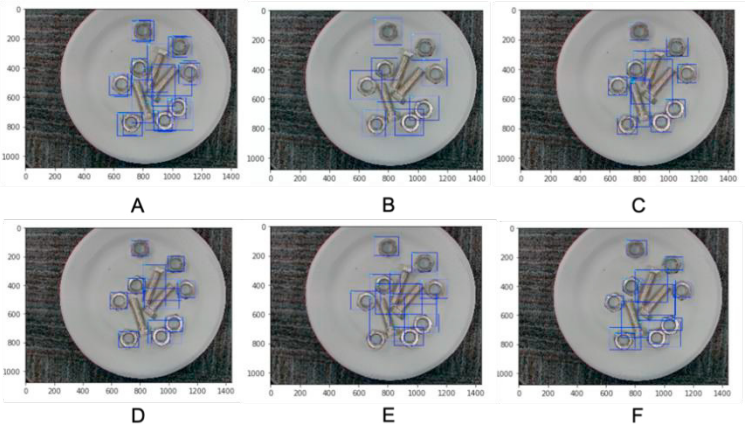

Fig. 5. Training results of different quantization cycles.

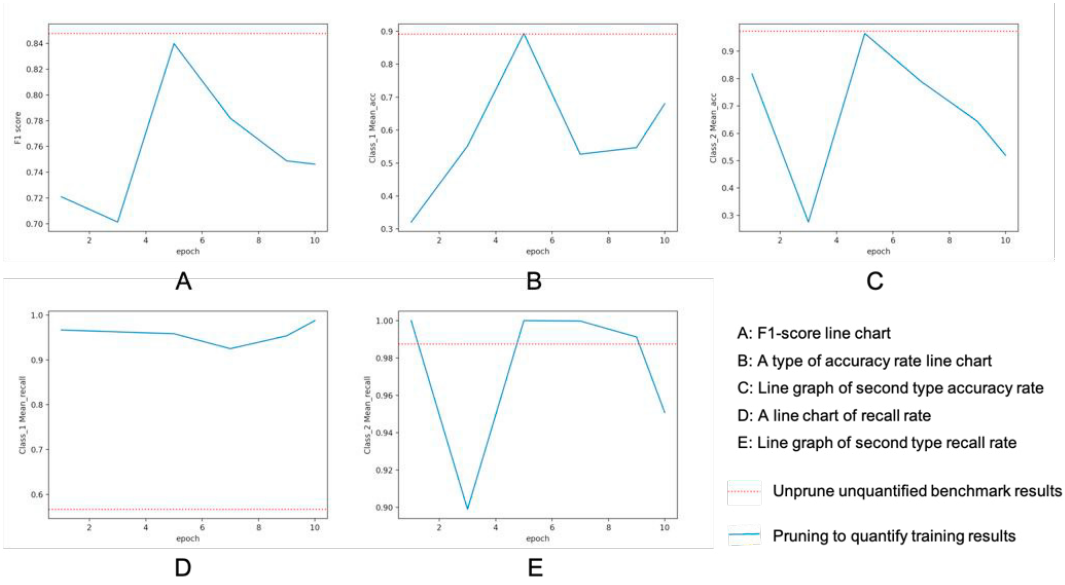

Fig. 6. Performance index chart.

Table 2. Accuracy and recall.

	1	3	5	7	9	10	Original
Class 1 accuracy	0.32	0.55	0.89	0.53	0.59	0.68	0.89
Class 2 accuracy	0.82	0.28	0.97	0.79	0.68	0.52	0.97
Class 1 recall rate	0.97	0.96	0.96	0.92	0.92	0.94	0.99
Class 2 recall rate	0.99	0.90	0.99	0.99	0.98	0.95	0.99

It can be seen that the F1 score and the accuracy curve of the first type are roughly in the shape of "increasing first and then decreasing". The actual meaning is: under a certain type of workpiece, when a smaller quantified training period is used or a larger quantized During the training cycle, the detection effect of such artifacts is relatively poor.

This can also be confirmed by the actual detection result graph. When the quantization training is 1 epoch and the quantization training is 10 epochs, it is difficult to detect long strip artifacts. At the same time, it can be seen that when quantizing training for 5 epochs, the F1 score and the accuracy of the first type can be approximated to the value without pruning and quantization, and the accuracy of the first type is even slightly higher than the situation without pruning and quantization. For the second type of accuracy curve, the fluctuation is more severe, showing the characteristic of "first down and then up" in turn.

Regarding the recall rate, it can be seen that the recall rate effect after quantitative pruning is partially improved or even greatly improved compared to the original effect. This is an unintended result of quantification and pruning. After analysis, it is speculated that it may be because the model avoids overfitting after being sufficiently thin. Compared with the original model, the complexity of the improved model is greatly reduced, and more potential candidate frames can be actively recalled.

At the same time, the forward inference Flops under different quantization training cycles and the model size saved after training are shown in the following table:

Table 3. Flops and model size.

	1	3	5	7	9	10	Original
Flops	6996514588.0	Same as 1	Same as 1	Same as 1	Same as 1	Same as 1	11702949888.0
Model size/MB	30.1	28.9	26.5	26.3	26.1	25.8	31.4

5. Conclusion and Outlook

Through experiments, it is found that on the basis of the MobileNet-Yolov3 network, proper model pruning and quantification[10,11] can greatly increase the speed of model detection while avoiding loss of accuracy, and reduce the number of model parameters, which is beneficial to the deployment of edge devices. In addition, in the process of quantifying and pruning the training scale, it is found that the period when the baseline model is overfitted can be used as the training reference period. After combining the actual specific quantization and pruning of the model, try to adjust the number of different quantized training periods. Experiments have proved that under this workpiece data set, training 5 quantization cycles and performing convolution kernel pruning on the three feature fusion branches of the original network according to the pruning ratio of [0.5, 0.4, 0.3], can be equivalent to the original model. At the same time, the model size has been reduced by nearly 20% and the forward inference speed is increased by 40%. In the future work, we will try to use more model compression and acceleration techniques to optimize more baseline models.

References

1. Redmon, Joseph, and Ali Farhadi. "Yolov3: An incremental improvement." arXiv preprint arXiv:1804.02767 (2018).
2. Howard, Andrew G., et al. "Mobilenets: Efficient convolutional neural networks for mobile vision applications." arXiv preprint arXiv:1704.04861 (2017).
3. Wen, Wei, et al. "Learning structured sparsity in deep neural networks." Advances in neural information processing systems. 2016.
4. Liu, Zhuang, et al. "Learning efficient convolutional networks through network slimming." Proceedings of the IEEE International Conference on Computer Vision. 2017.
5. Li, Hao, et al. "Pruning filters for efficient convnets." arXiv preprint arXiv:1608.08710 (2016).
6. Miyashita, Daisuke, Edward H. Lee, and Boris Murmann. "Convolutional neural networks using logarithmic data representation." arXiv preprint arXiv:1603.01025 (2016).
7. Krishnamoorthi, Raghuraman. "Quantizing deep convolutional networks for efficient inference: A whitepaper." arXiv preprintarXiv:1806.08342 (2018).
8. Liu Xuejian. Research on the Acceleration Method of Neural Network Reasoning for Target Detection[D]. Shandong University, 2020.
9. Gong Kaiqiang, Zhang Chunmei, Zeng Guanghua. Convolutional neural network model pruning combined with tensor decomposition compression method[J/OL]. Computer Applications
10. Liu Leshan. Research on Algorithm Optimization of Convolutional Neural Network Model Compression[D]. Hebei University of Economics and Business, 2020.
11. Chen Zhen. Research on acceleration, compression and evaluation methods of deep neural networks[D]. University of Science and Technology of China, 2019.